T10 Doors and Keys

CRE132 worksheet

What you'll make

Three little puzzles, each guarding a reward in a fenced pen: a **locked door** that needs the gold key from T09, a **gate** held open by a **pressure plate**, and a gate worked by a **lever**.

The door uses the inventory: a key is just an item. The plate and the lever don't open anything themselves. They send **Unity Events** (as in S09), and you connect them to the gate in the Inspector. The gate never knows what opened it, which keeps every piece small and reusable.



Figure 1: The finished scene: three fenced pens with a locked door, a pressure plate and crate, and a lever.

Before you start

1. Unzip `CRE132_T10_DoorsAndKeys_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T10_DoorsAndKeys_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T10_DoorsAndKeys`.
3. Press **Play**. You can walk (WASD or arrows), attack (J) and pick up the gold key in the meadow: it shows in the inventory (T09). You can push the crate. The door, the gates, the plate and the lever are solid scenery for now.

The puzzle pieces are under **Puzzles** in the Hierarchy: **Locked Door**, two objects called **Gate**, **Pressure Plate** and **Lever**. Their sprites and colliders are already set up. For the lever you'll add an **Interact** action; T11 uses it for talking too.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. A door that needs a key: LockedDoor

Make a folder `Assets/_game/Scripts/T10_DoorsAndKeys` and create `LockedDoor` in it:

```
// A door that opens when the player bumps into it carrying the right key (T09's
// inventory). The key is used up. Without it, the door stays shut and says what it needs.
using UnityEngine;

public class LockedDoor : MonoBehaviour
{
    [SerializeField] private ItemData key;
    [SerializeField] private Sprite openSprite;

    private SpriteRenderer spriteRenderer;
    private Collider2D doorCollider;
    private bool isOpen;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        doorCollider = GetComponent<Collider2D>();
    }

    // The door's collider is solid, so walking into it is a collision, not a trigger.
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (isOpen)
        {
            return;
        }

        Inventory inventory = collision.gameObject.GetComponent<Inventory>();
        if (inventory == null)
        {
            return;
        }

        // Remove returns true only if there was a key to take.
        if (inventory.Remove(key))
        {
            Open();
        }
        else
        {
            Debug.Log("Locked. You need a " + key.itemName + ".");
        }
    }

    private void Open()
    {
        isOpen = true;
        spriteRenderer.sprite = openSprite;
        // No collider: the player can walk through.
        doorCollider.enabled = false;
    }
}
```

- **`OnCollisionEnter2D`, not a trigger:** the door has to block you, so its collider is solid. Walking into a solid collider is a collision.
- **`inventory.Remove(key)` does two jobs at once:** it answers "did you have a key?" and, if so, uses it up. That's why T09 made `Remove` return a `bool`.
- **Opening** swaps the sprite and switches the collider off, so you can walk through.

Add it to **Puzzles > Locked Door**. Drag the **Gold Key** asset (`Assets/_game/ScriptableObjects/Items/`) into **Key** and the `door_open` sprite (`Assets/_game/Art/Sprites/Kenney/`) into **Open Sprite**.

Test it: walk into the door: the Console says `Locked. You need a Gold Key.` Fetch the key from the meadow and walk into the door again. It opens, and the key disappears from the inventory.

2. A gate anything can open: Gate

Create `Gate` in the same folder:

```
// A gate that something else opens and closes: a pressure plate, a lever. It has no idea
// what controls it; they call Open, Close or Toggle through their Unity Events (S09).
using UnityEngine;

public class Gate : MonoBehaviour
{
    [SerializeField] private Sprite closedSprite;
    [SerializeField] private Sprite openSprite;
    [SerializeField] private bool startOpen = false;

    private SpriteRenderer spriteRenderer;
    private Collider2D gateCollider;
    private bool isOpen;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        gateCollider = GetComponent<Collider2D>();
        SetOpen(startOpen);
    }

    public void Open()
    {
        SetOpen(true);
    }

    public void Close()
    {
        SetOpen(false);
    }

    public void Toggle()
    {
        SetOpen(!isOpen);
    }

    private void SetOpen(bool open)
    {
        isOpen = open;
        gateCollider.enabled = !open;
        if (open)
        {
            spriteRenderer.sprite = openSprite;
```

```
        }
        else
        {
            spriteRenderer.sprite = closedSprite;
        }
    }
}
```

- **Three public methods, one private worker.** `Open`, `Close` and `Toggle` all call `SetOpen`, so the sprite and collider are only changed in one place.
- **`!isOpen`** means "not isOpen": `Toggle` flips the gate whichever way it is.
- **Nothing here mentions plates or levers.** Anything that can call a public method can open this gate.

Add it to both **Gate** objects under **Puzzles** (select both, Add Component). Set **Closed Sprite** to `gate_closed` and **Open Sprite** to `gate_open`, both from `Art/Sprites/Kenney/`. Leave **Start Open** unticked.

3. Stand on it: PressurePlate

Create `PressurePlate` in the same folder:

```
// A plate that's pressed while anything solid stands on it: the player, or a pushed
// crate. It doesn't open anything itself: its On Pressed and On Released events are wired
// up in the Inspector (to a Gate's Open and Close, for example).
using UnityEngine;
using UnityEngine.Events;

public class PressurePlate : MonoBehaviour
{
    [SerializeField] private Sprite upSprite;
    [SerializeField] private Sprite downSprite;
    [SerializeField] private UnityEvent onPressed = new UnityEvent();
    [SerializeField] private UnityEvent onReleased = new UnityEvent();

    private SpriteRenderer spriteRenderer;

    // How many things are on the plate. Counting (instead of a true/false) means stepping
    // off doesn't release it while a crate is still there.
    private int thingsOnPlate;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.sprite = upSprite;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        // Ignore other triggers, such as the sword's hitbox or a pickup.
        if (other.isTrigger)
        {
            return;
        }

        thingsOnPlate++;
        if (thingsOnPlate == 1)
        {
```

```
            spriteRenderer.sprite = downSprite;
            onPressed.Invoke();
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.isTrigger)
        {
            return;
        }

        thingsOnPlate--;
        if (thingsOnPlate == 0)
        {
            spriteRenderer.sprite = upSprite;
            onReleased.Invoke();
        }
    }
}
```

- **It counts what's on it.** With a simple true/false, stepping off would release the plate even with a crate still on it. With a count, it's only released when the count gets back to 0.
- **`other.isTrigger`** skips other triggers, such as the sword's hitbox, so swinging at the plate doesn't press it.
- **`UnityEvent`** fields show up in the Inspector as lists you can add calls to. **`Invoke()`** runs whatever you put there.

Add it to **Puzzles > Pressure Plate**. Set **Up Sprite** to `plate_up` and **Down Sprite** to `plate_down`. Then connect it to the gate in the same pen, the **Gate** at Position X **19.5** (check its Transform):

1. Under **On Pressed**, click **+**. Drag that **Gate** into the slot that says **None (Object)**, then open the **No Function** menu and choose **Gate > Open**.
2. Do the same under **On Released**, choosing **Gate > Close**.

Test it: stand on the plate: it goes down and the gate opens. Step off and it shuts before you can get there. Now push the **Crate** onto the plate and walk through.

> **The crate has no script.** It's a **Dynamic** Rigidbody 2D (**Mass** 3, **Linear Damping** 10, **Freeze Rotation Z** ticked) with a Box Collider 2D. The player's own physics pushes it, and the damping stops it sliding when you stop.

4. Pull it: Lever

Create `Lever` in the same folder:

```
// A lever the player pulls by standing next to it and pressing Interact (E, gamepad
// North). Each pull flips it, and its On Switched event tells whatever it's wired to (a
// Gate's Toggle).
using UnityEngine;
using UnityEngine.Events;
using UnityEngine.InputSystem;

public class Lever : MonoBehaviour
{
    [SerializeField] private InputActionReference interactAction;
    [SerializeField] private Sprite offSprite;
```

```
    [SerializeField] private Sprite onSprite;
    [SerializeField] private UnityEvent onSwitched = new UnityEvent();

    private SpriteRenderer spriteRenderer;
    private bool isOn;
    // True while the player is inside the lever's trigger (close enough to reach it).
    private bool playerInReach;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.sprite = offSprite;
    }

    private void OnEnable()
    {
        interactAction.action.Enable();
    }

    private void OnDisable()
    {
        interactAction.action.Disable();
    }

    private void Update()
    {
        if (playerInReach && interactAction.action.WasPressedThisFrame())
        {
            isOn = !isOn;
            if (isOn)
            {
                spriteRenderer.sprite = onSprite;
            }
            else
            {
                spriteRenderer.sprite = offSprite;
            }
            onSwitched.Invoke();
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            playerInReach = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            playerInReach = false;
        }
    }
}
```

- **The trigger marks “close enough”.** The Lever’s Circle Collider 2D (radius 1, **Is Trigger** ticked) sets `playerInReach` when the player walks in and clears it when they walk out.

- **Input is read in `Update`**, because trigger messages only arrive when something enters or leaves. Pressing E only counts while you're in reach.
- **One event, `On Switched`**, because the lever flips each time. Wired to `Gate.Toggle`, the gate flips too.

First, an Interact action. Double-click `CRE132Controls` (in `Assets/_game/Input`) and add an action called `Interact` (**Button**) with two bindings: **E [Keyboard]** and **Button North [Gamepad]** (Y on Xbox, Triangle on PlayStation). Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor.

Add it to **Puzzles > Lever**. In **Interact Action**, pick **Player/Interact** (from `CRE132Controls`). Set **Off Sprite** to `lever_off` and **On Sprite** to `lever_on`. Under **On Switched**, click **+**, drag in the **Gate** at Position X **27.5** and choose **Gate > Toggle**.

Test it: walk up to the lever and press **E**: it flips and the gate opens. Press E again: it shuts. Pressing E from far away does nothing.

Check your work

- ☐ `CRE132Controls` has an **Interact** action bound to E and Button North.
- ☐ The door says it's locked until you have the gold key, then opens and uses the key up.
- ☐ Standing on the plate opens its gate; stepping off shuts it. The crate on the plate keeps it open.
- ☐ E next to the lever opens and shuts its gate. E from far away does nothing.
- ☐ Hitting the plate with your sword doesn't press it.
- ☐ Compare with `CRE132_T10_DoorsAndKeys_Finished` if something behaves differently.

Key points

- **Keys are just items.** The door asks the inventory, and `Remove` both checks for the key and uses it in one go.
- **The thing that's triggered (the gate) doesn't know about the trigger** (the plate or the lever). Unity Events join them in the Inspector, so one plate could open two gates, or a gate and a sound, with no new code.
- **Count, don't just remember true or false**, when several things can be on a switch at once.

If something goes wrong

- **The door never opens, even with the key:** its **Key** slot is a different asset, or empty. It must be the same **Gold Key** asset the pickup gives.
- **NullReferenceException in LockedDoor:** the **Key** slot is empty (the message uses `key.itemName`).
- **The gate opens but stays invisible, or shows the wrong picture:** a sprite slot on the **Gate** is empty or swapped.
- **The plate goes down but nothing opens:** the On Pressed list has no call, or it points at the wrong **Gate**. Check the function says **Gate.Open**.
- **The wrong gate opens:** both gates are called **Gate**. Check the Position X of the one you dragged in.
- **NullReferenceException in Lever:** the **Interact Action** slot is empty.
- **E does nothing next to the lever:** the Player must be tagged **Player**, and the Lever's collider must have **Is Trigger** ticked.

Try it

1. Make the lever gate start open: tick **Start Open**. What does the first pull do now?
2. A timed lever: the gate closes again after 3 seconds. (Hint: `Invoke("Close", 3f)` in Gate.)
3. A red key and a red door: a second `ItemData` asset, a pickup prefab for it and a second door.
4. A gate that needs two plates pressed at once: a small script with two counters, or a Gate that only opens when a count reaches 2.

5. Show “Locked” above the door in the world instead of in the Console (T11’s talk prompt is the same idea).